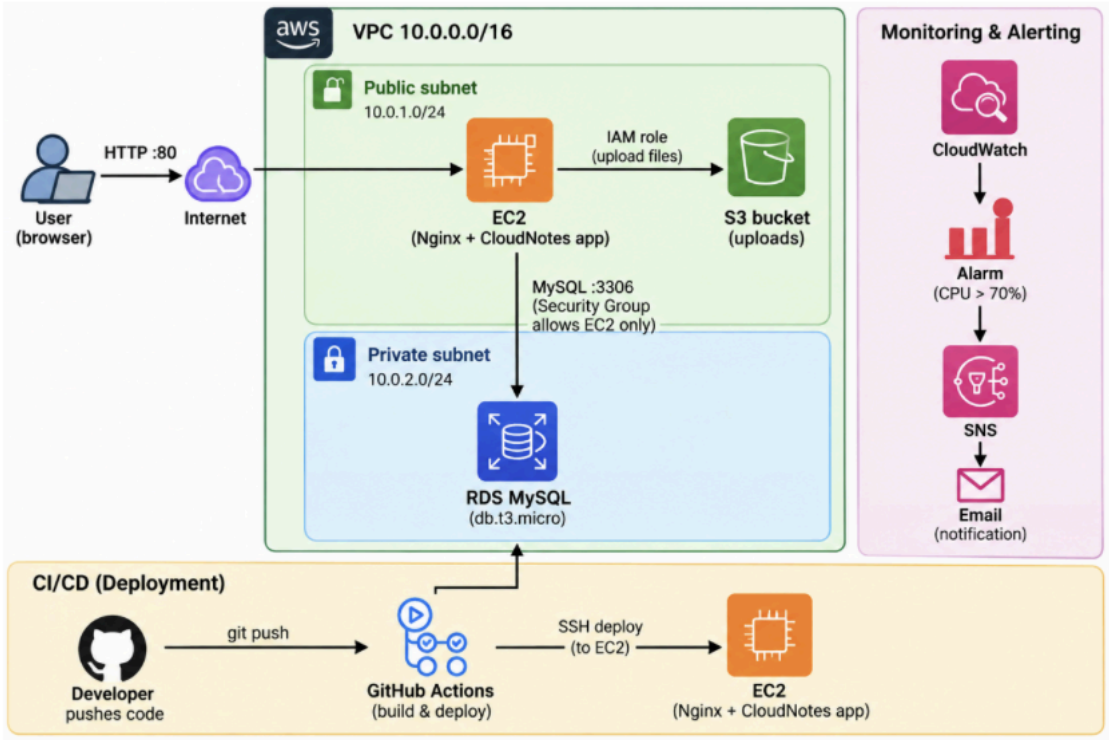
AWS Infrastructure Documentation — DevNotes

Architecture



1. IAM (Identity and Access Management)

Purpose: IAM helps manage access to AWS resources by controlling identities, permissions, and authentication.

Screenshot 1 — Root User Security Credentials

What I did: Reviewed the AWS root user's security credentials, MFA configuration, and access-key status.

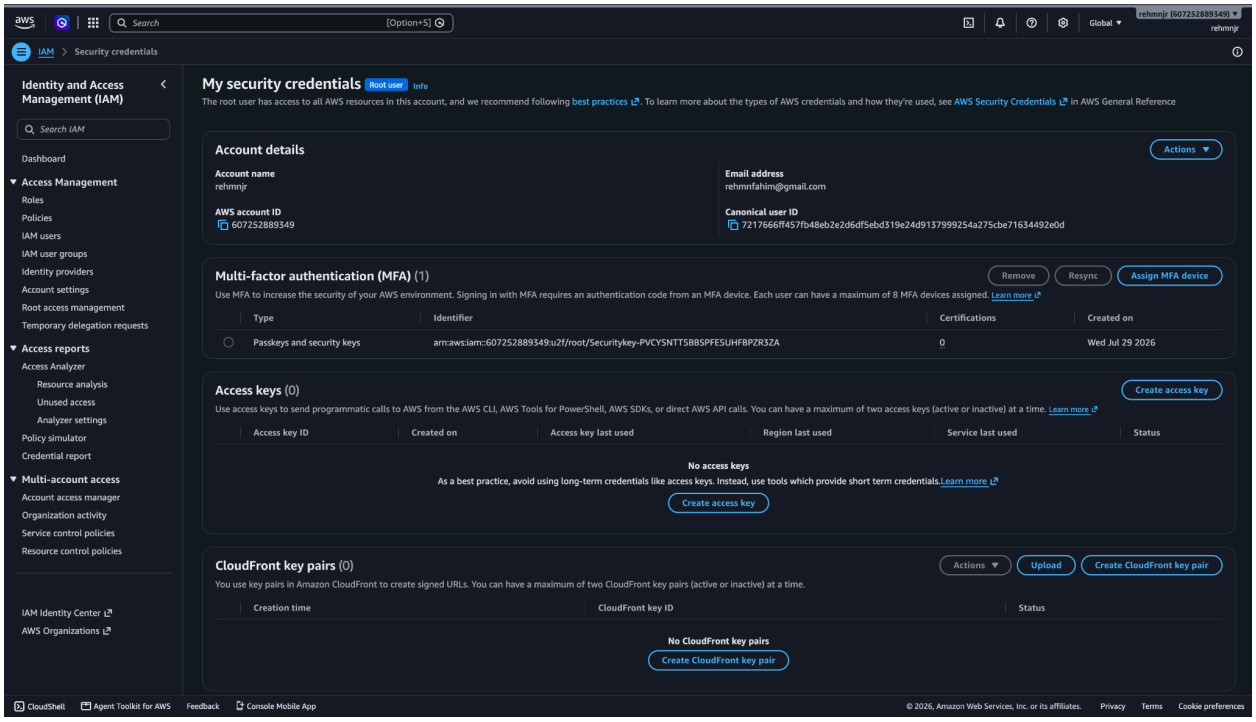
Why did it: To verify account-level authentication security and check whether long-term root access keys were present.

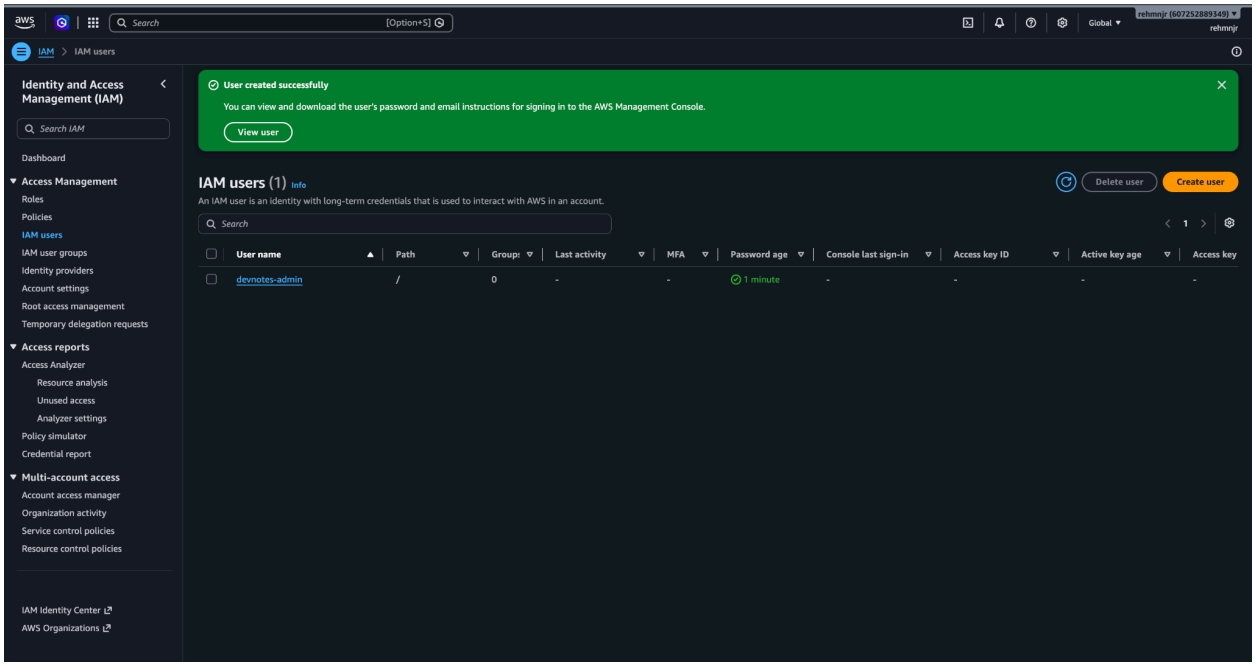
Configuration:

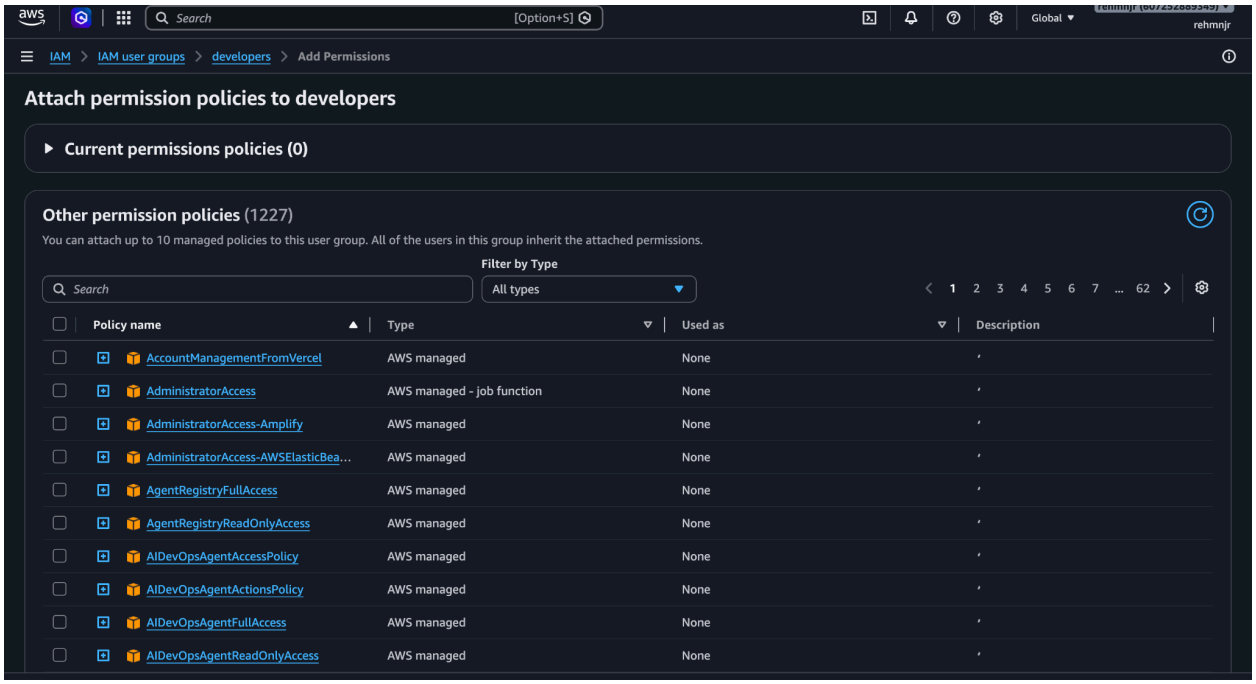
- Multi-factor authentication (MFA): A passkey/security key is listed.
- Root user access keys: None listed.
- CloudFront key pairs: None listed.

Result: MFA is configured for the root user, and no root access keys are listed in this screenshot.

Security note: The root user should be reserved for tasks that specifically require root access. Use an IAM identity with appropriate permissions for routine administration. Never publish account identifiers or sensitive credential details unnecessarily.







IAM User Group and Permissions

What I did: Created an IAM user group named `developers` and attached the AWS-managed `AmazonEC2ReadOnlyAccess` policy.

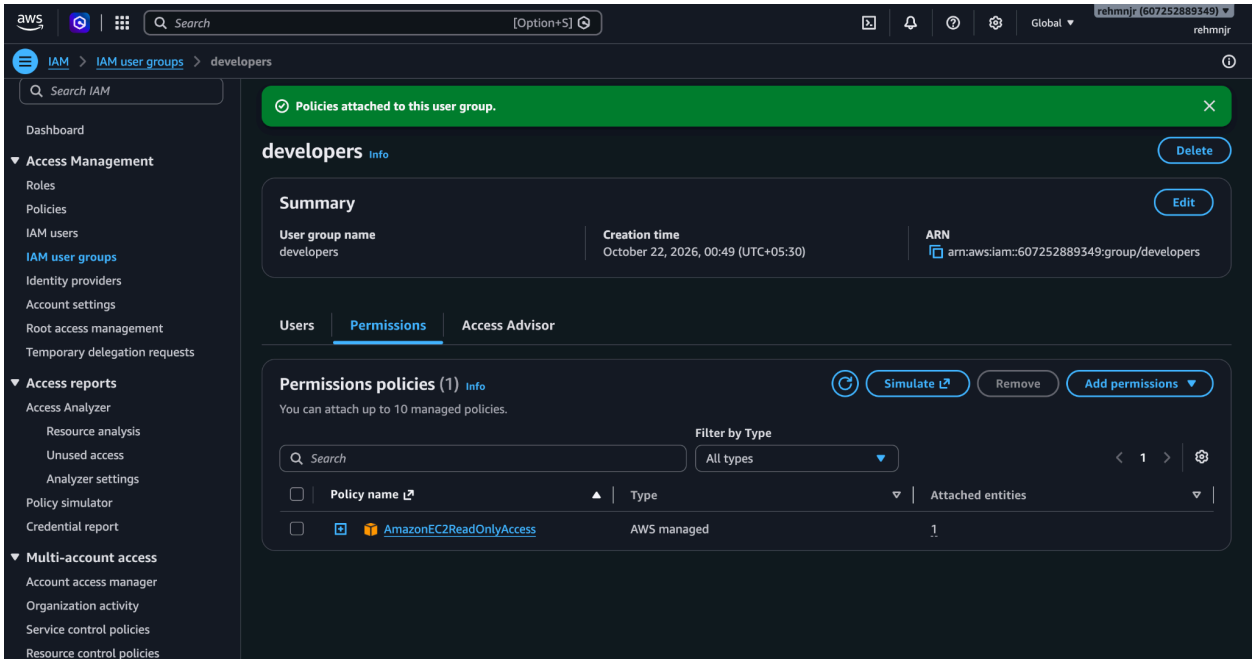
Why did it: To organize developer identities and manage permissions at the group level instead of assigning the same permissions individually to every user.

Configuration:

- **Group name:** `developers`
- **Attached policy:** `AmazonEC2ReadOnlyAccess`
- **Policy type:** AWS managed
- **Purpose:** Read-only access to supported EC2 resources.

Result: The policy was successfully attached to the group.

Security note: This policy does not grant permission to create, modify, or terminate EC2 instances. Group membership and the permissions of individual users should be reviewed separately.



Screenshot 2 — EC2 IAM Role

What I did: Created an IAM role named `DevNotes-EC2-S3-Role` trusted by the Amazon EC2 service.

Why did it: To allow the EC2 instance to access permitted AWS services without storing long-term AWS access keys on the server.

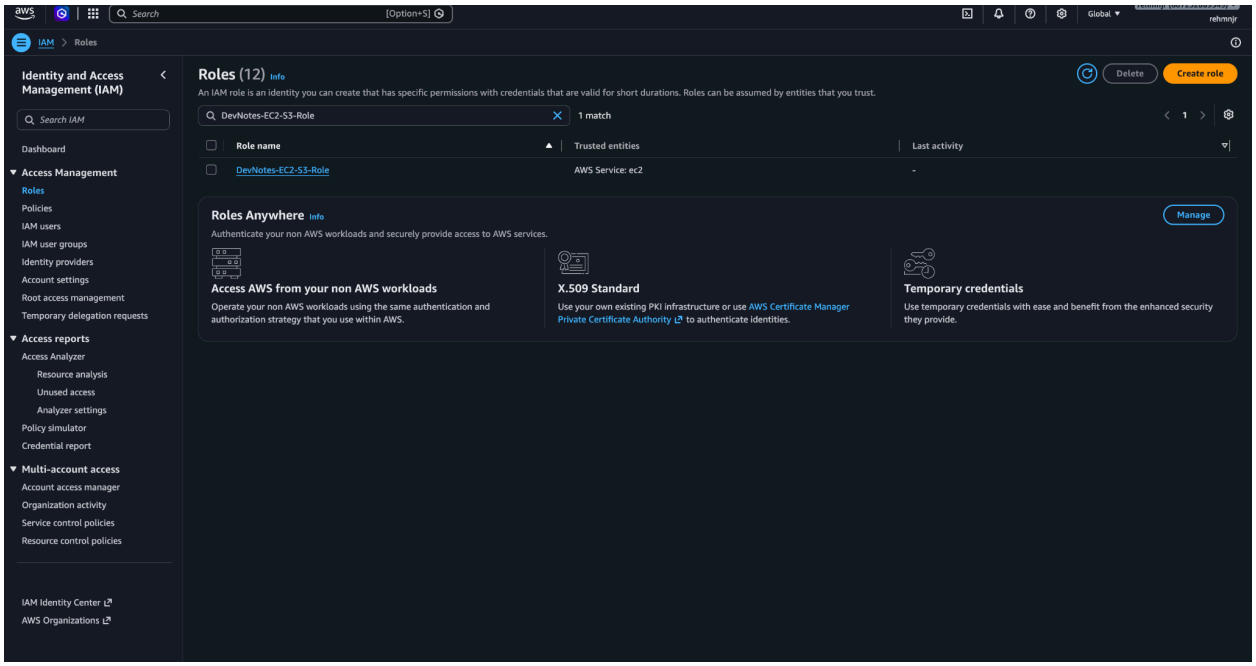
Configuration:

- **Role name:** `DevNotes-EC2-S3-Role`

- **Trusted entity:** AWS Service — EC2
- **Intended use:** Grant the DevNotes EC2 instance permissions to access its required AWS resources, including S3 backups.

Result: The IAM role is present in AWS and configured to trust the EC2 service.

Note: This screenshot confirms the role and its trusted service, but does not show the attached policies. The actual S3 permissions must be verified in the role's **Permissions** tab.



---

2. AWS Budgets — Cost Monitoring

Screenshot 3 — Monthly Cost Budget Configuration

What I did: Configured a monthly cost budget named `DevNotes-Monthly-Budget`.

Why did it: To receive email notifications when AWS spending reaches or is forecast to exceed the configured budget.

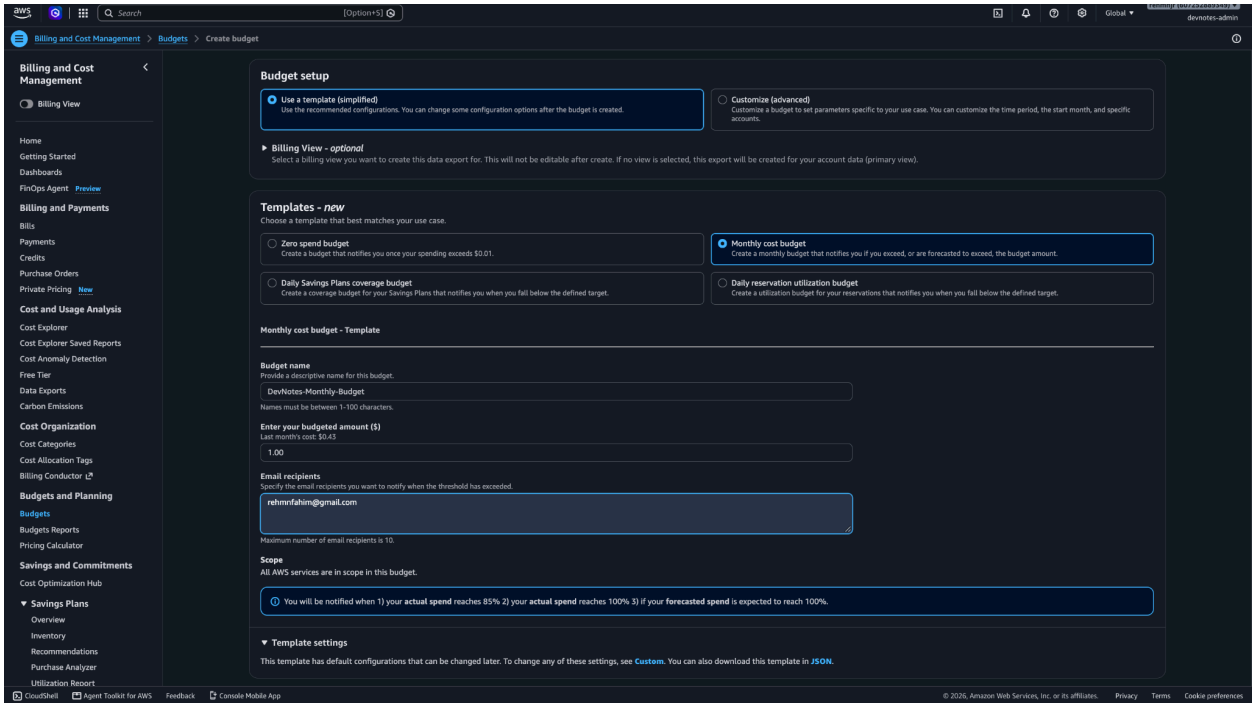
Configuration:

- **Budget name:** `DevNotes-Monthly-Budget`
- **Budget type:** Monthly cost budget
- **Budget amount:** $1.00
- **Scope:** All AWS services

- **Notification recipient:** Configured email address
- **Alert thresholds:** 85% actual spending, 100% actual spending, and forecasted spending reaching 100%.

Result: The budget configuration is designed to alert the account owner about project costs.

Important: This screenshot shows the budget setup form, not proof that the budget was successfully created or that notifications were delivered. AWS Budgets sends alerts; it does not automatically stop resources when the threshold is exceeded.



---

3. Amazon VPC — Network Infrastructure

Screenshot 4 — DevNotes VPC

What I did: Created a dedicated Virtual Private Cloud (VPC) named `DevNotes-VPC-vpc`.

Why did it: To isolate the application's AWS networking resources and control communication between the web server and database.

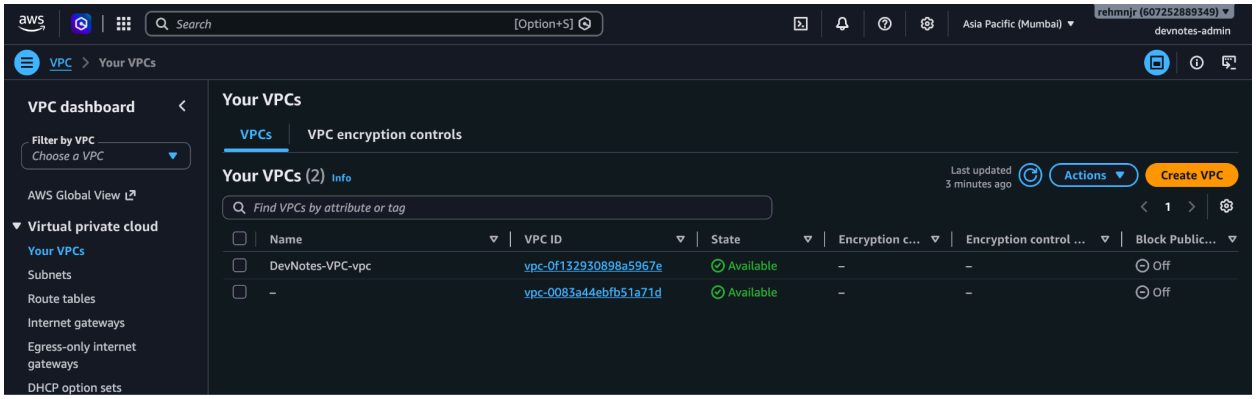
Configuration:

- **VPC name:** `DevNotes-VPC-vpc`
- **VPC ID:** `vpc-0f132930898a5967e`

- **Status:** Available
- **Region:** Asia Pacific (Mumbai), `ap-south-1`

Result: The dedicated VPC is available for the DevNotes infrastructure.

Note: The screenshot shows two VPCs in the account. Only the named DevNotes VPC is identified here as part of this project; the other VPC should not be assumed to be project-related.



Screenshot 5 — VPC Subnets Configuration

What I did: Configured subnets within the `DevNotes-VPC-vpc` VPC, including two private subnets and a dedicated public subnet.

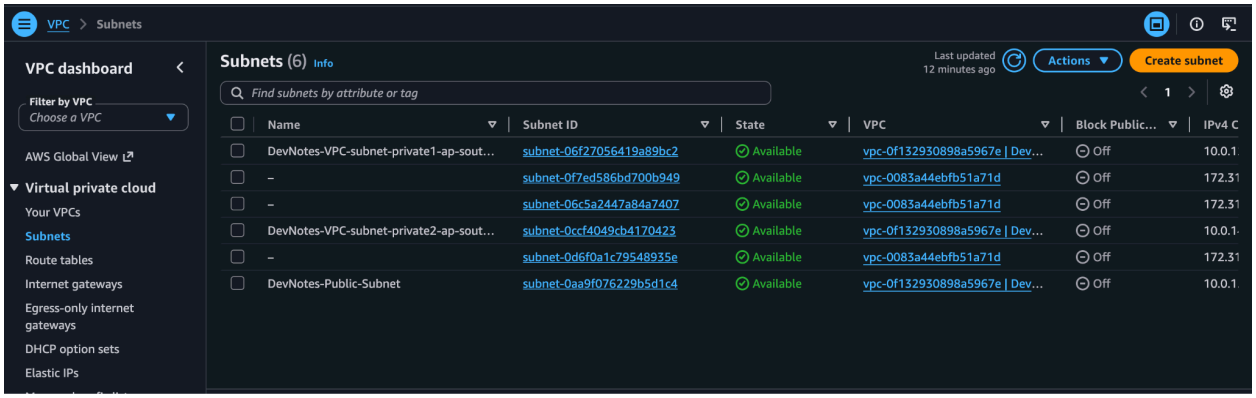
Why did it: To organize AWS resources into separate network segments and support controlled communication between the application server and database.

Configuration:

- **VPC:** `DevNotes-VPC-vpc`
- **Private subnets:** Two project subnets are listed.
- **Public subnet:** `DevNotes-Public-Subnet`
- **Status:** All listed subnets show `Available`.
- **Additional subnets:** The screenshot also shows subnets belonging to another VPC.

Result: The DevNotes VPC has separate public and private subnet resources.

Note: The screenshot truncates the CIDR ranges and does not show route-table associations. Public accessibility must be verified using the subnet's route table and public IP configuration, not just its name.



Screenshot 6 — VPC Route Tables

What I did: Configured separate route tables for the DevNotes VPC, including route tables associated with the two private subnets.

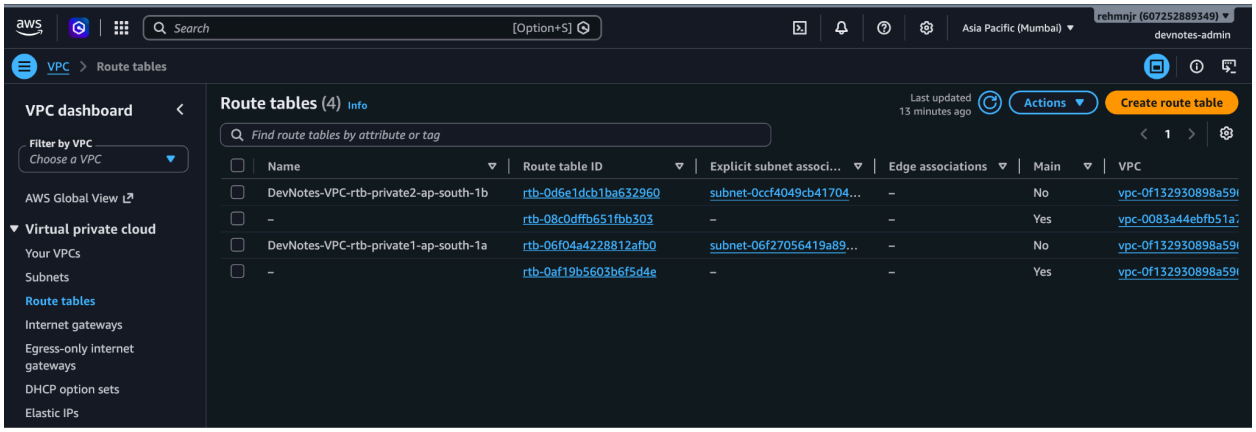
Why did it: Route tables determine where network traffic is directed, allowing public and private subnets to follow different routing rules.

Configuration:

- **Private route table 1:** `DevNotes-VPC-rtb-private1-ap-south-1a`
- **Private route table 2:** `DevNotes-VPC-rtb-private2-ap-south-1b`
- **Subnet associations:** The two named private route tables show explicit subnet associations.
- **Additional route tables:** The screenshot also shows main route tables for the DevNotes VPC and another VPC.

Result: Separate route tables are available for the DevNotes private subnets.

Note: This list view does not show the destination routes. The route table's **Routes** tab must be checked to confirm whether a default route points to an Internet Gateway, NAT Gateway, or another target.



Screenshot 7 — Internet Gateway Configuration

What I did: Created an Internet Gateway named `DevNotes-IGW` and attached it to `DevNotes-VPC-vpc`.

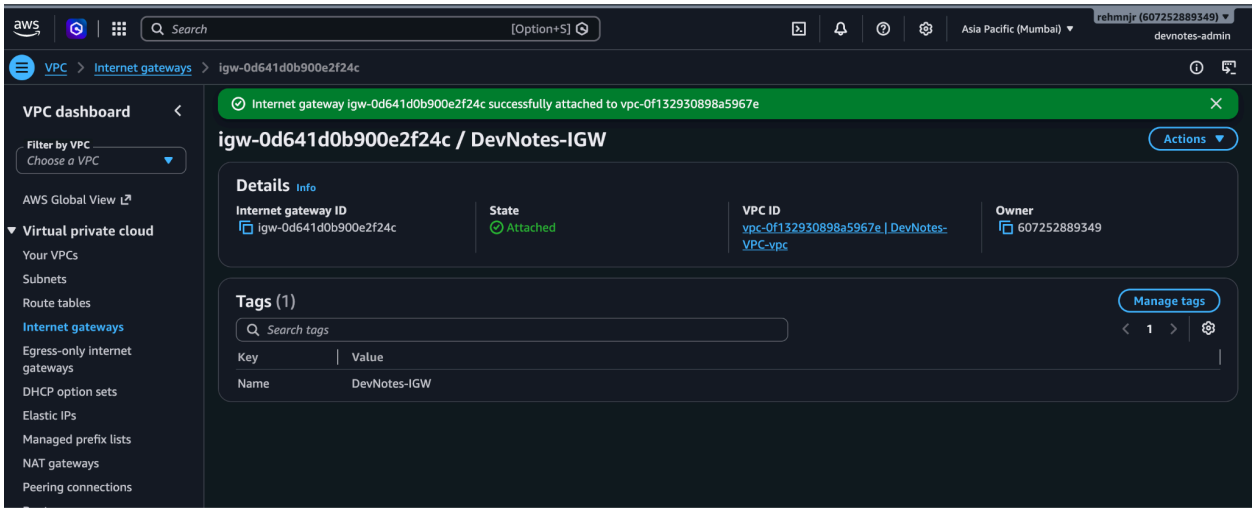
Why did it: An Internet Gateway enables internet connectivity for resources in the VPC when the appropriate routes, public IP addressing, and security rules are configured.

Configuration:

- **Name:** `DevNotes-IGW`
- **Gateway ID:** `igw-0d641d0b900e2f24c`
- **Attached VPC:** `DevNotes-VPC-vpc`
- **Status:** Attached

Result: The Internet Gateway is successfully attached to the DevNotes VPC.

Important: Attaching an Internet Gateway alone does not make a subnet public. The route table must include a route such as `0.0.0.0/0` pointing to this gateway, and the instance must have suitable public addressing and security-group rules.



Screenshot 8 — EC2 Web Server Security Group

What I did: Created a security group named `DevNotes-Web-SG` for the DevNotes web server.

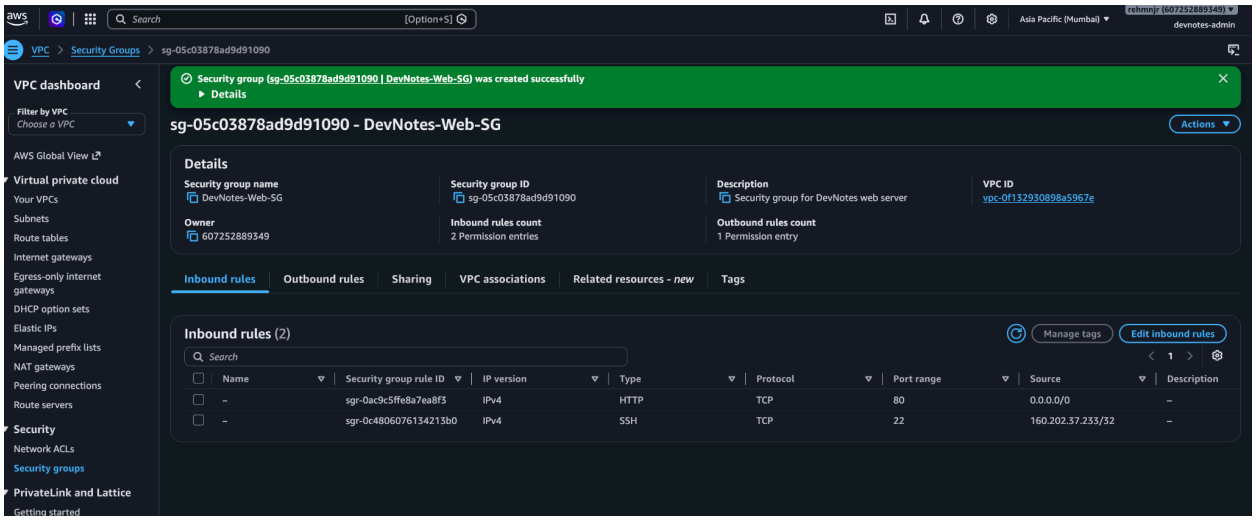
Why did it: To control incoming network traffic to the EC2 instance.

Configuration:

Protocol	Port	Source	Purpose
HTTP	80	`0.0.0.0/0`	Allows public HTTP access to the web application.
SSH	22	`160.202.37.233/32`	Restricts SSH access to one specific public IPv4 address.

Result: The security group was created successfully with two inbound rules.

Security note: HTTP on port 80 is publicly accessible. SSH is restricted to a single IP, which is safer than allowing SSH from `0.0.0.0/0`. Confirm that the SSH source IP remains current and trusted.



---

4. Amazon EC2 — Application Server

Screenshot 9 — DevNotes EC2 Instance Running

What I did: Launched an EC2 instance named `DevNotes-Web-Server` to host the containerized application.

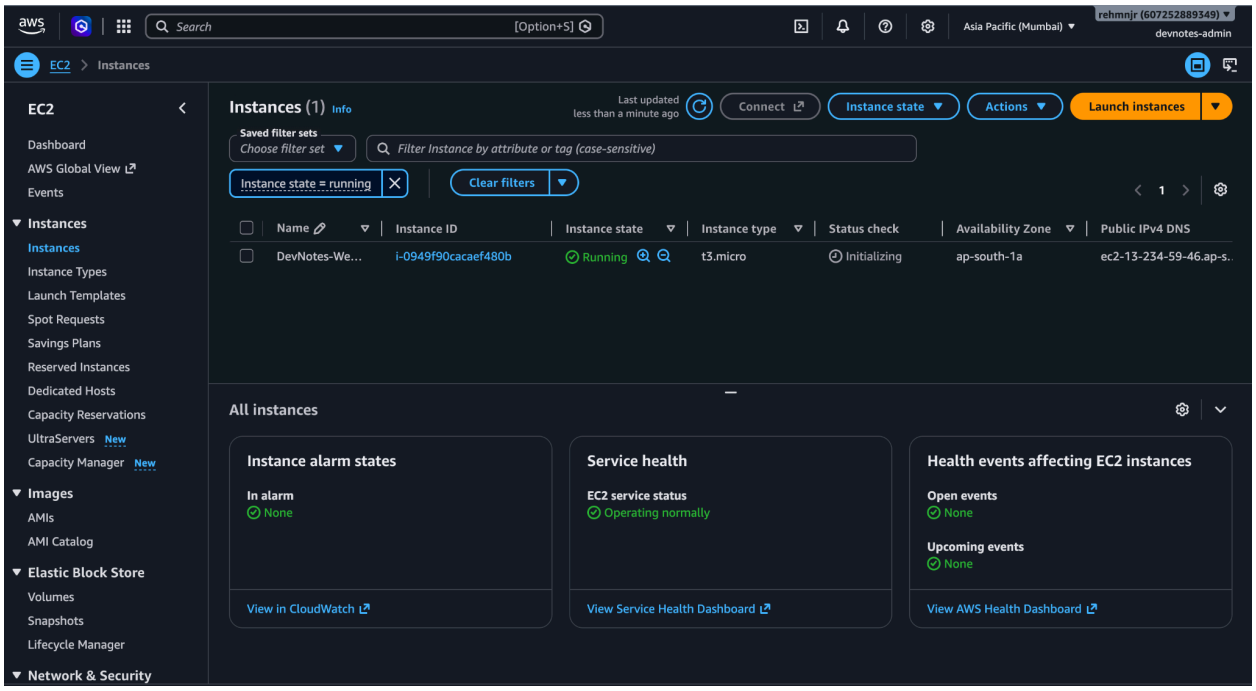
Why did it: To run the DevNotes frontend and backend containers on a cloud-based virtual server.

Configuration:

- **Instance ID:** `i-0949f90cacaef480b`
- **Instance type:** `t3.micro`
- **Availability Zone:** `ap-south-1a`
- **Instance state:** Running
- **AWS Region:** Mumbai (`ap-south-1`)

Result: The EC2 instance was running when this screenshot was captured.

Note: The instance status check was still initializing in this screenshot. A running instance does not by itself prove that the application is healthy; application and API health checks must also pass.



5. Amazon S3 — Backup Storage

Screenshot 10 — Create S3 Bucket

What I did: Created an S3 bucket named `devnotes-attachments-607252889349`.

Why did it: To store DevNotes database backup files separately from the EC2 application server, helping preserve backup copies if the server is replaced or terminated.

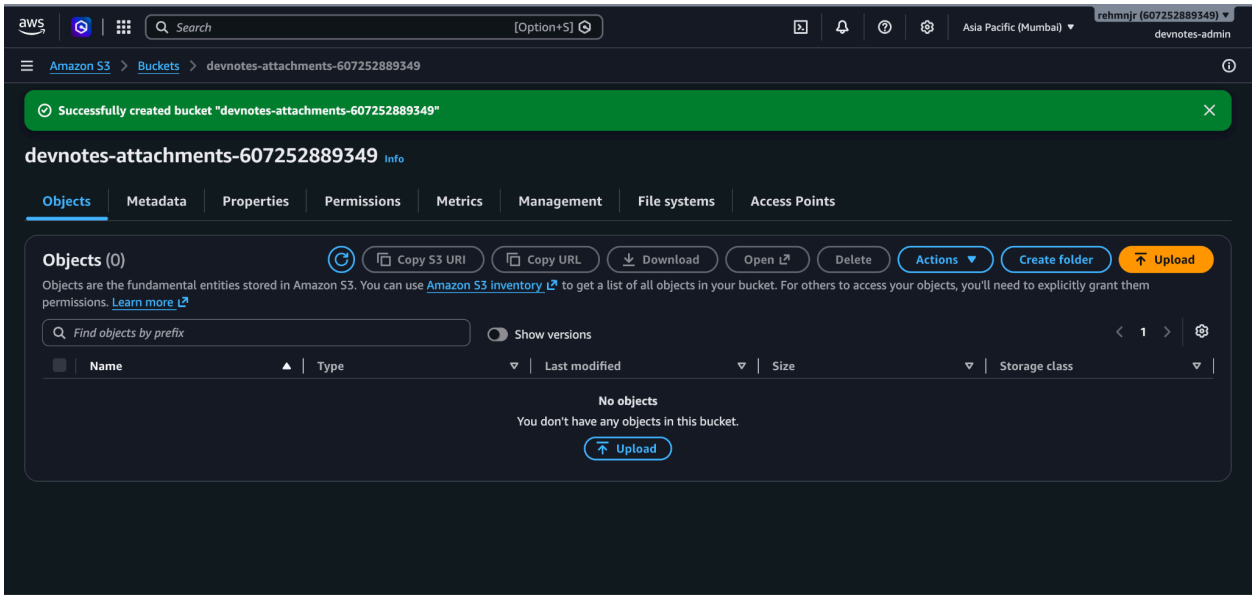
Configuration:

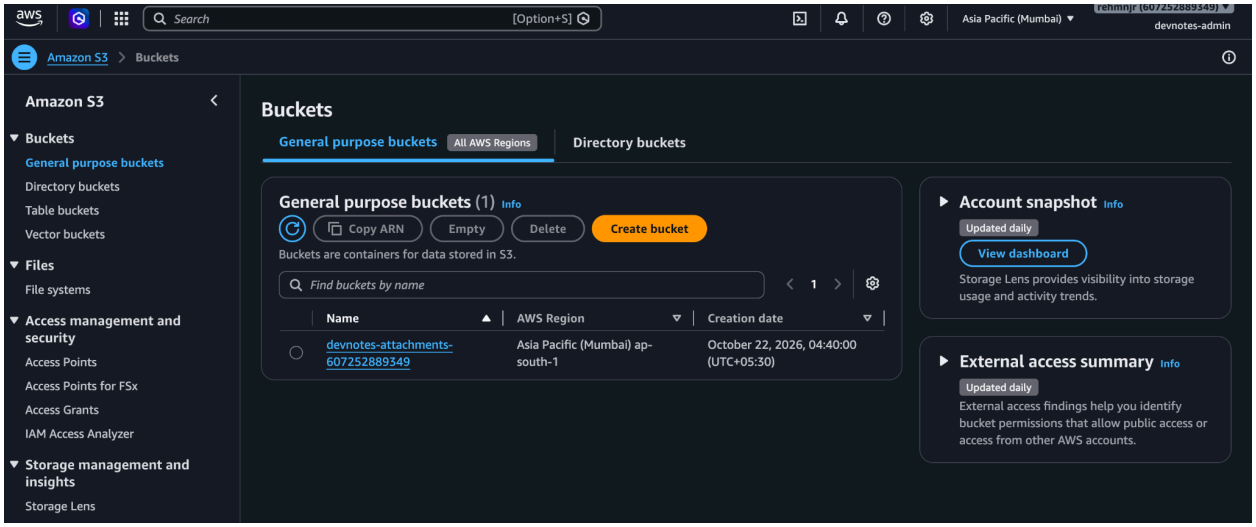
- **Bucket name:** `devnotes-attachments-607252889349`
- **AWS Region:** Asia Pacific (Mumbai), `ap-south-1`
- **Bucket creation:** Successful
- **Objects shown:** devnotes-attachments-607252889349

Result: The S3 bucket was successfully created and was initially empty.

Next configuration: The bucket was later used to store database backup files under the `backups/` prefix. Two backup files, each approximately 12 KB, were subsequently visible in that folder.

Note: This screenshot does not show the bucket's versioning, encryption, or public-access-block settings. Those should be documented from the bucket's Properties and Permissions tabs if screenshots are available.





Screenshot 11 — SSH Connection and Nginx Installation

What I did: Connected to the EC2 instance using SSH and the `DevNotes-Key.pem` key pair. Updated the Amazon Linux 2023 package metadata and installed Nginx.

Why did it: SSH provided administrative access to the server, while Nginx was installed to serve HTTP traffic and support reverse-proxy routing.

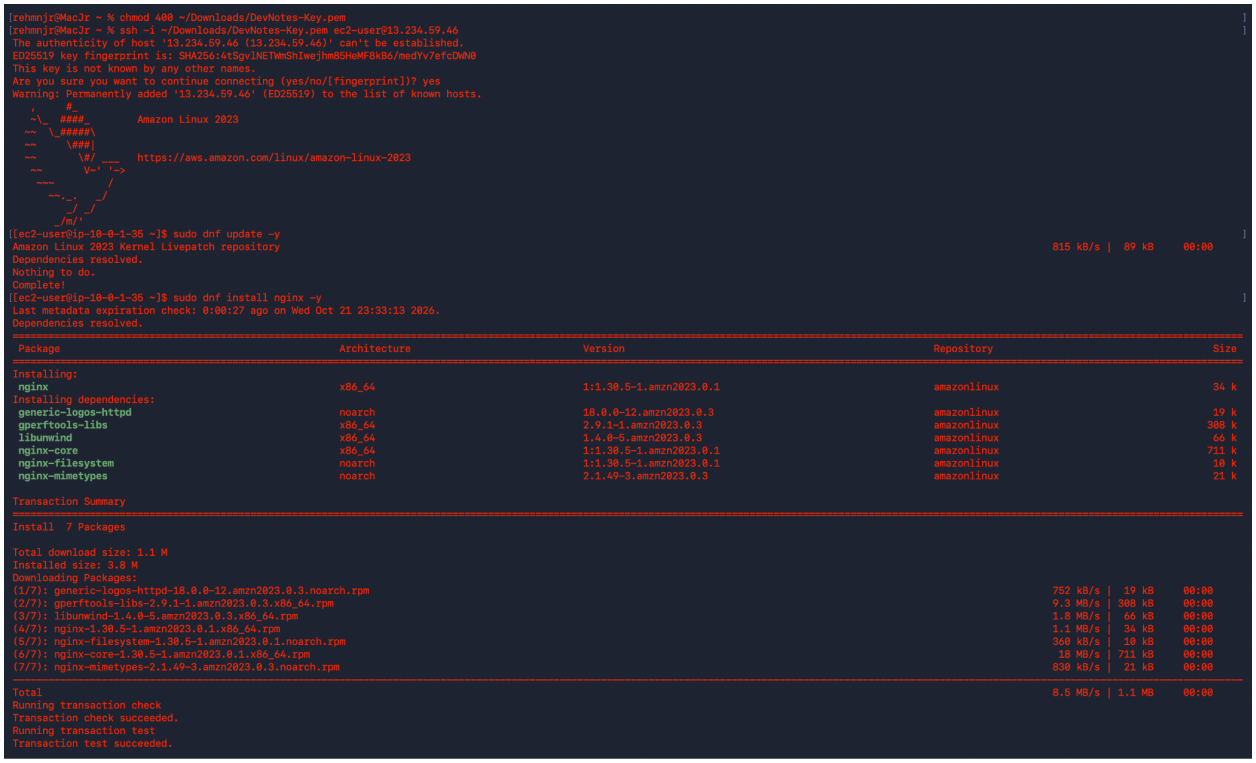
Configuration:

- **Operating system:** Amazon Linux 2023
- **SSH user:** `ec2-user`
- **SSH key:** `DevNotes-Key.pem`
- **Package manager:** `dnf`
- **Web server:** Nginx

Commands used:

chmod 400 ~/Downloads/DevNotes-Key.pem
ssh -i ~/Downloads/DevNotes-Key.pem ec2-user@13.234.59.46
sudo dnf update -y
sudo dnf install nginx -y

Result: The SSH session was established, and Nginx was installed successfully.



Screenshot 12 — Nginx Service Verification

What I did: Enabled Nginx to start automatically at boot and started the service immediately.

Why did it: To ensure that the web server runs as a managed system service and is available after a server restart.

Command used:

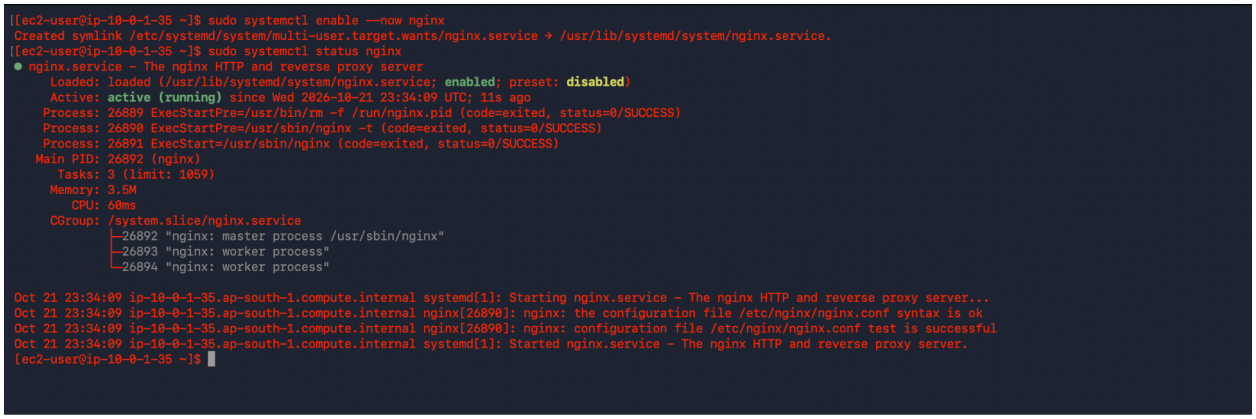
sudo systemctl enable --now nginx
sudo systemctl status nginx

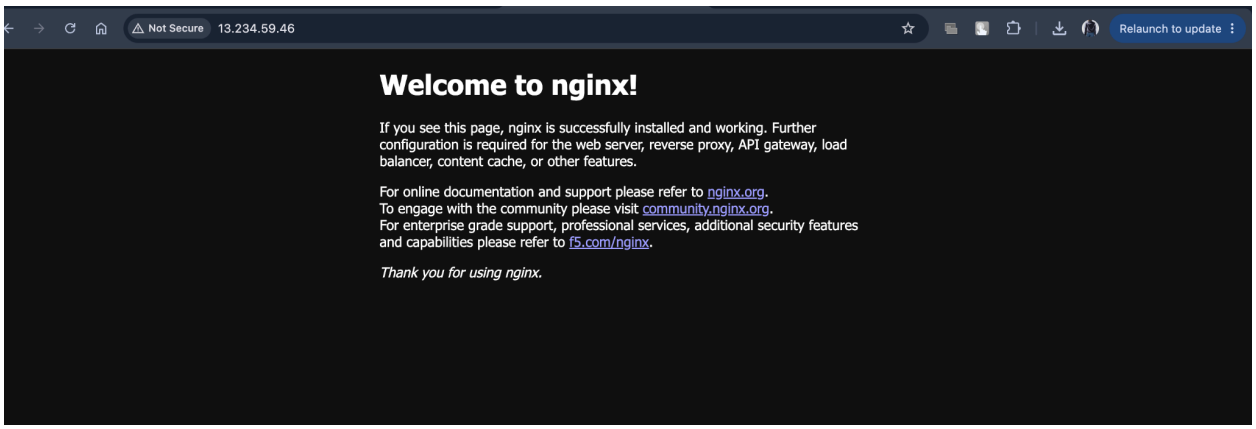
Configuration:

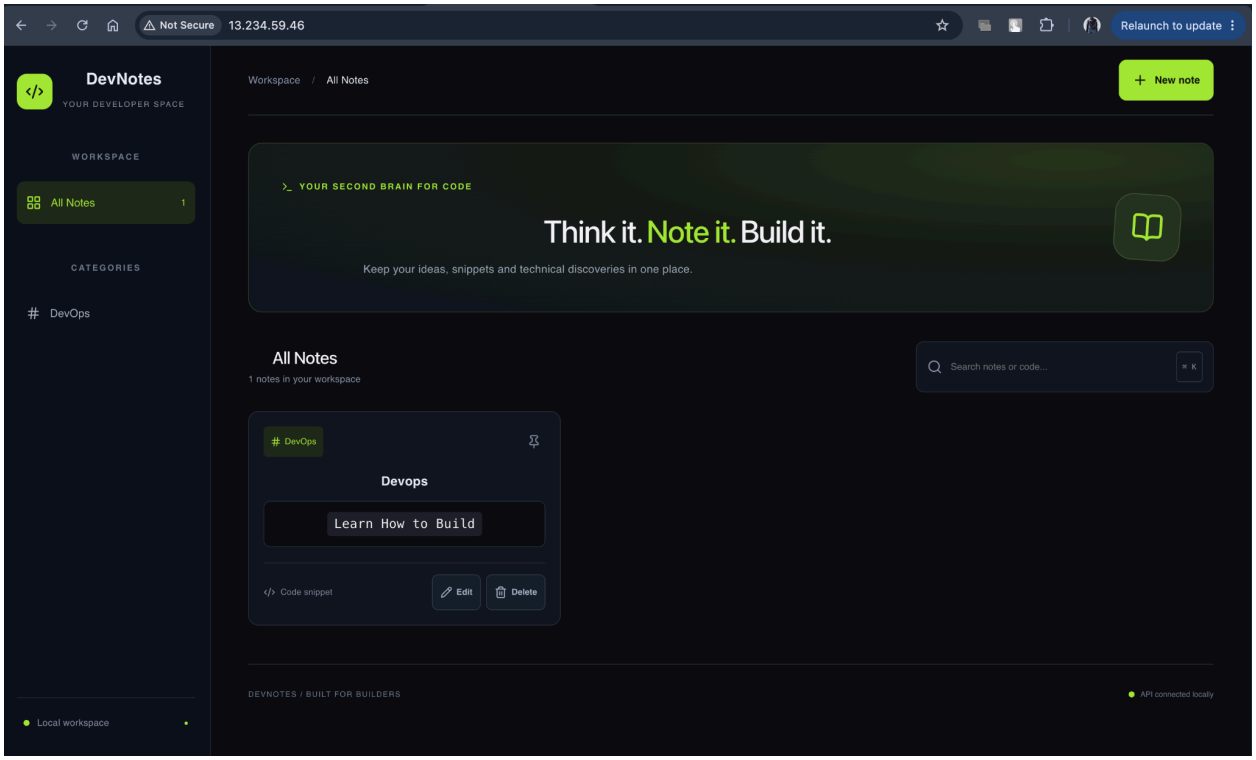
- **Service:** `nginx.service`
- **Enabled at boot:** Yes
- **Current status:** Active (running)
- **Configuration test:** Successful

Result: The screenshot confirms that Nginx was running and its configuration syntax check succeeded.

Note: This verifies the Nginx service at that point in time; it does not by itself prove that the complete DevNotes application or backend API was working.







6. Amazon CloudWatch — Monitoring and Alerts

Screenshot 13 — EC2 CPU Utilization Alarm

What I did: Created a CloudWatch alarm named `DevNotes-High-CPU-Alarm` to monitor the EC2 instance's CPU utilization.

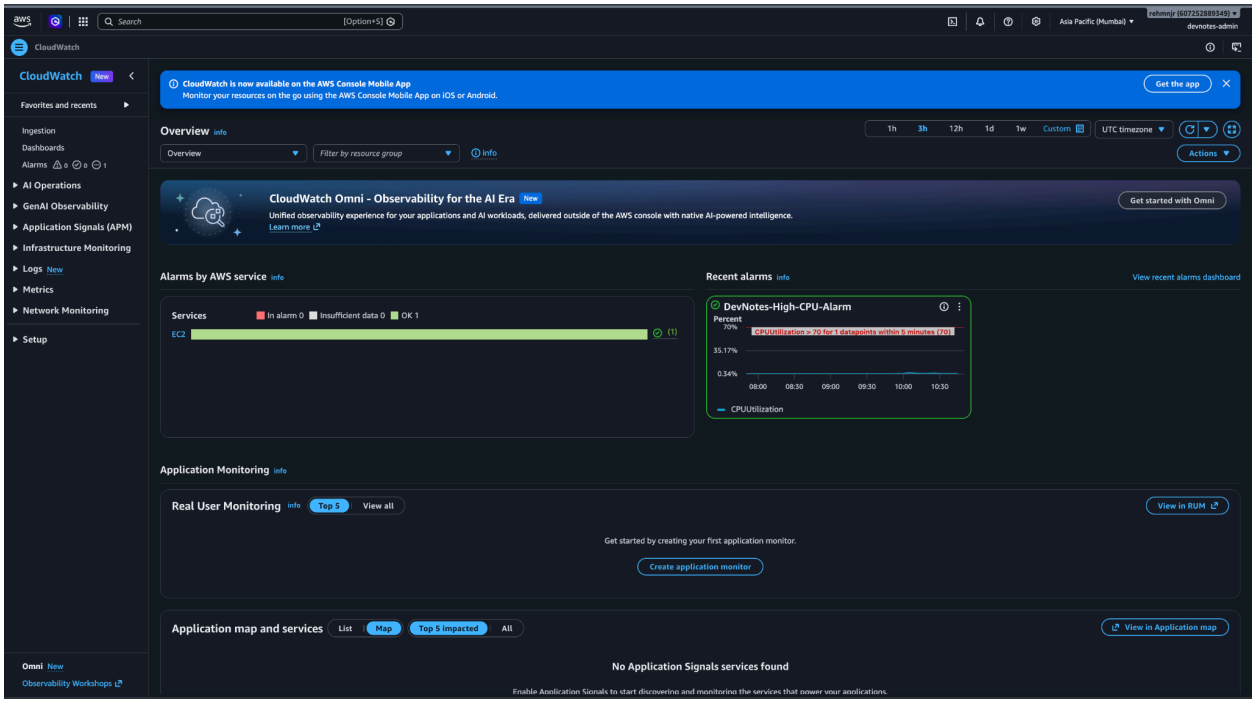
Why did it: To identify unusually high CPU usage and receive alerts when the server exceeds the configured threshold.

Configuration:

- **Alarm name:** `DevNotes-High-CPU-Alarm`
- **Monitored metric:** `CPUUtilization`
- **Threshold:** Greater than 70%
- **Evaluation period:** 1 datapoint within 5 minutes
- **Current state shown:** OK
- **Insufficient data alarms:** 0

Result: CloudWatch displays the alarm in the OK state, meaning the configured alarm condition was not currently breached based on the available evaluation data.

Note: This screenshot confirms the alarm and its threshold. It does not independently confirm that an email notification was delivered or that a high-CPU test successfully triggered the alarm.



Amazon SNS — CloudWatch Alert Notifications

Amazon Simple Notification Service (SNS) was configured to provide email notifications for the DevNotes project's CloudWatch monitoring alerts.

Screenshot 14 — SNS Topic and Email Subscription

What I did: Created the Standard SNS topic `fahim-project2-alerts` and configured an email subscription.

Why did it: To provide a notification destination for CloudWatch alarms when their configured actions are triggered.

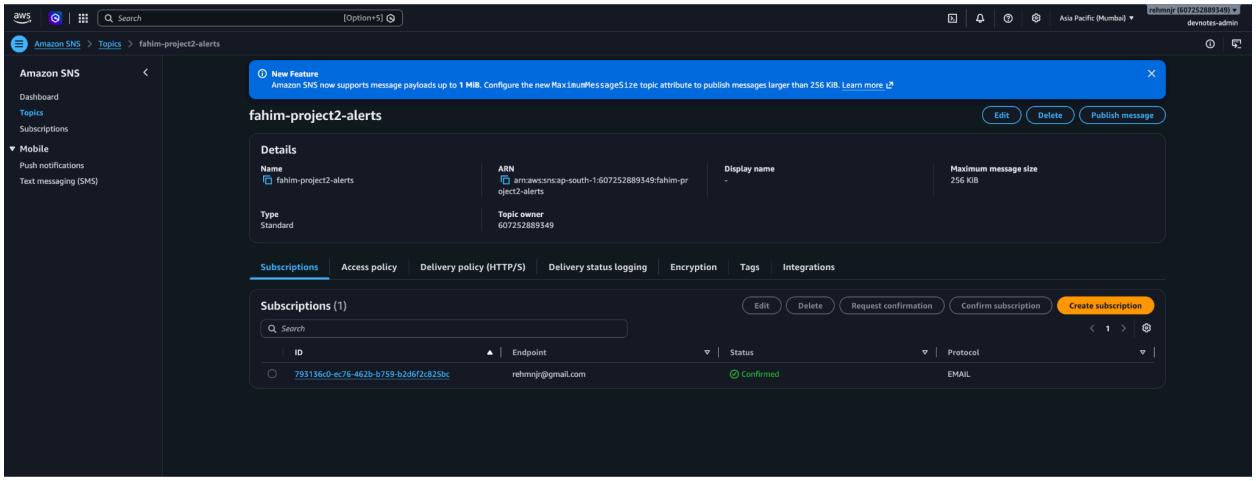
Configuration:

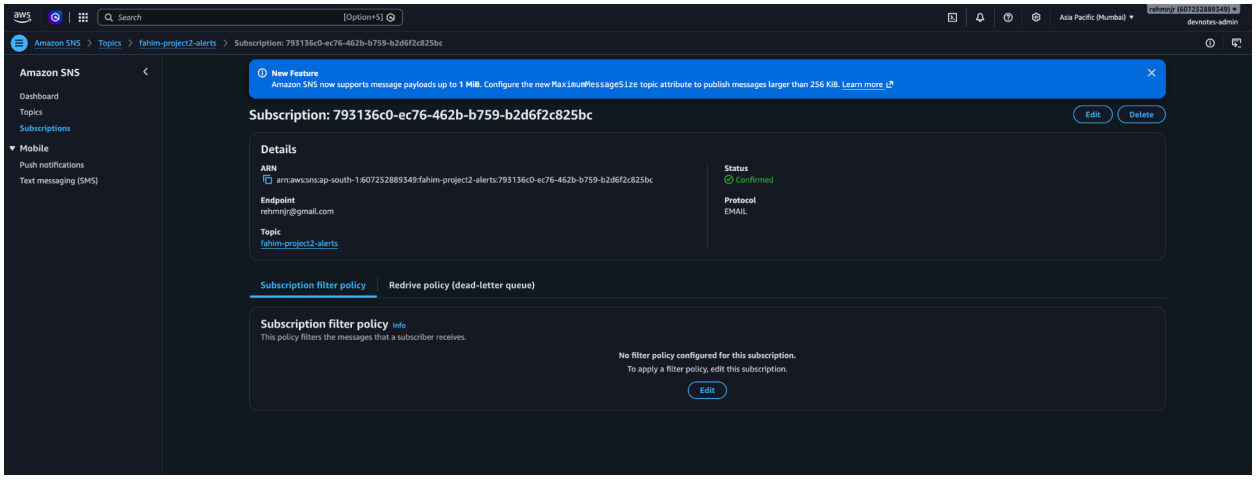
- **Topic name:** `fahim-project2-alerts`

- **Topic type:** Standard
- **Region:** Mumbai (`ap-south-1`)
- **Email endpoint:** `rehmnjr@gmail.com`
- **Subscriptions:** 1
- **Subscription status:** Confirmed

Result: The SNS topic and its email subscription were configured successfully.

Note: The screenshots confirm the topic and subscription status. They do not independently prove that an alarm notification was delivered to the inbox.





Amazon CloudWatch — EC2 CPU Monitoring

Screenshot 15 — DevNotes High CPU Alarm Configuration

What I did: Created a metric alarm named `DevNotes-High-CPU-Alarm` to monitor CPU utilization of the DevNotes EC2 instance.

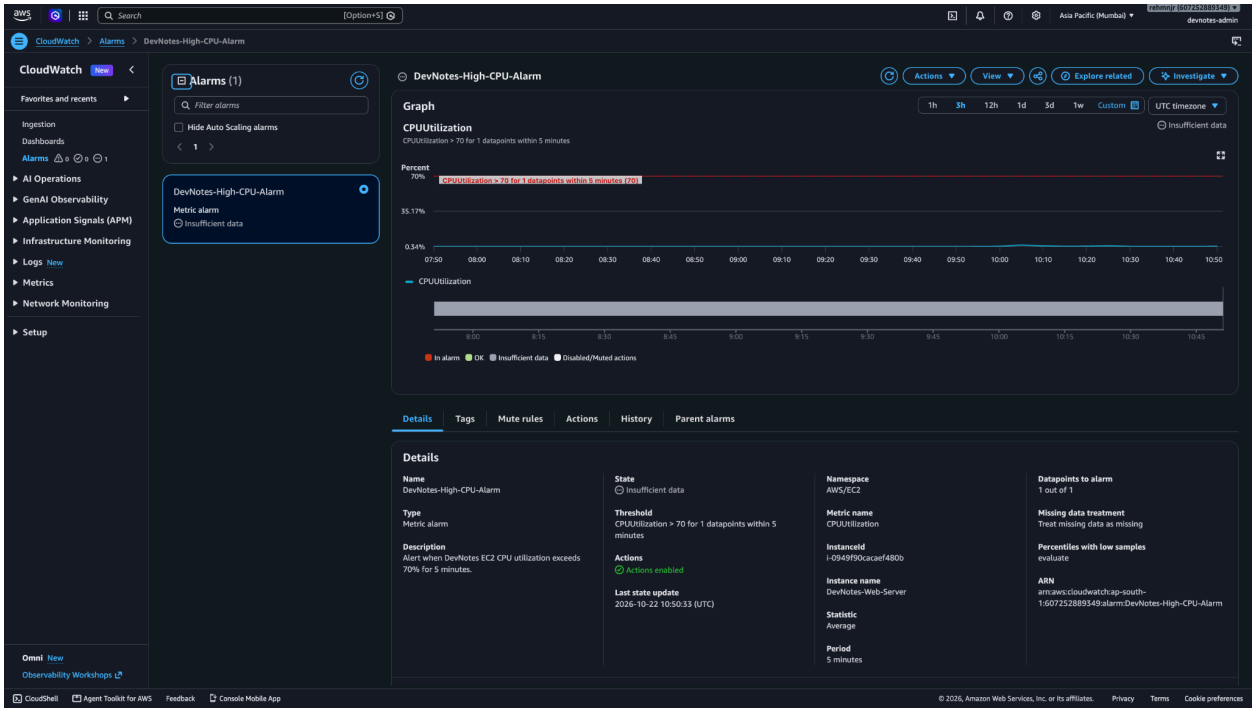
Why did it: To detect high CPU usage and enable notifications when the server's resource consumption exceeds the configured threshold.

Configuration :

- **Alarm name:** `DevNotes-High-CPU-Alarm`
- **EC2 instance:** `DevNotes-Web-Server`
- **Instance ID:** `i-0949f90cacaef480b`
- **Namespace:** `AWS/EC2`
- **Metric:** `CPUUtilization`
- **Statistic:** Average
- **Threshold:** CPU utilization greater than 70%
- **Period:** 5 minutes
- **Datapoints to alarm:** 1 out of 1
- **Missing data treatment:** Treat missing data as missing
- **Alarm actions:** Enabled
- **State shown:** Insufficient data

Result: The alarm was configured with a 70% CPU threshold and a five-minute evaluation period. At the time of this screenshot, CloudWatch displayed `Insufficient data`, so the alarm's current evaluation could not be confirmed.

Note: Alarm actions were enabled. The SNS topic and confirmed email subscription were configured separately to support email notifications.



IAM — EC2 Role and Custom Policies

What I did: Created the IAM role `DevNotes-EC2-S3-Role` and attached two custom inline policies.

Why did it: To allow the EC2 application server to access required AWS resources through an IAM role instead of storing long-term AWS access keys on the server.

Configuration:

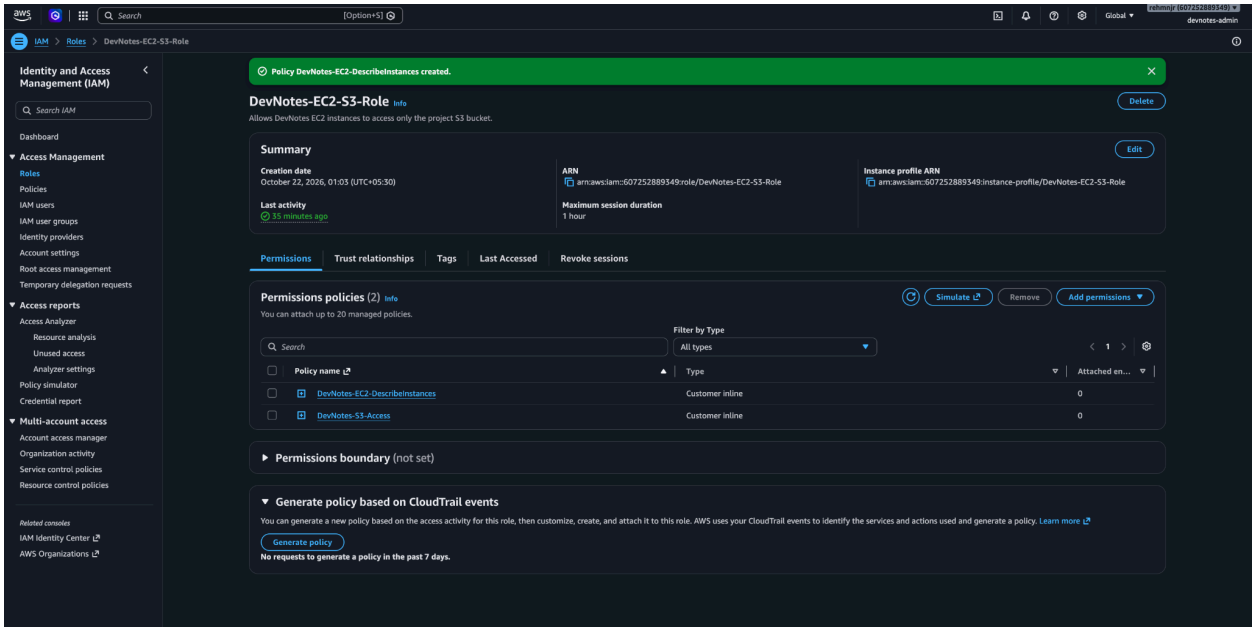
- **Role name:** `DevNotes-EC2-S3-Role`
- **Trusted service:** EC2 (as established in the role configuration)
- **Instance profile:** `DevNotes-EC2-S3-Role`
- **Maximum session duration:** 1 hour
- **Inline policy 1:** `DevNotes-S3-Access`
- **Inline policy 2:** `DevNotes-EC2-DescribeInstances`
- **Permissions boundary:** Not set

Purpose of the policies:

- `DevNotes-S3-Access` — grants the S3 permissions defined in the policy for the project's backup bucket.
- `DevNotes-EC2-DescribeInstances` — grants permission to describe EC2 instances.

Result: The IAM role and both inline policies were present in the AWS console.

Security note: The exact actions and resource scope must be verified by opening each policy. The role screenshot alone does not show the full policy JSON or prove that every permission is restricted to the intended resources.



Amazon S3 — Database Backup Verification

What I did: Stored two database backup files in the `backups/` folder of the DevNotes S3 bucket.

Why did it: To keep backup copies separate from the EC2 instance, so they can be recovered even if the application server is terminated.

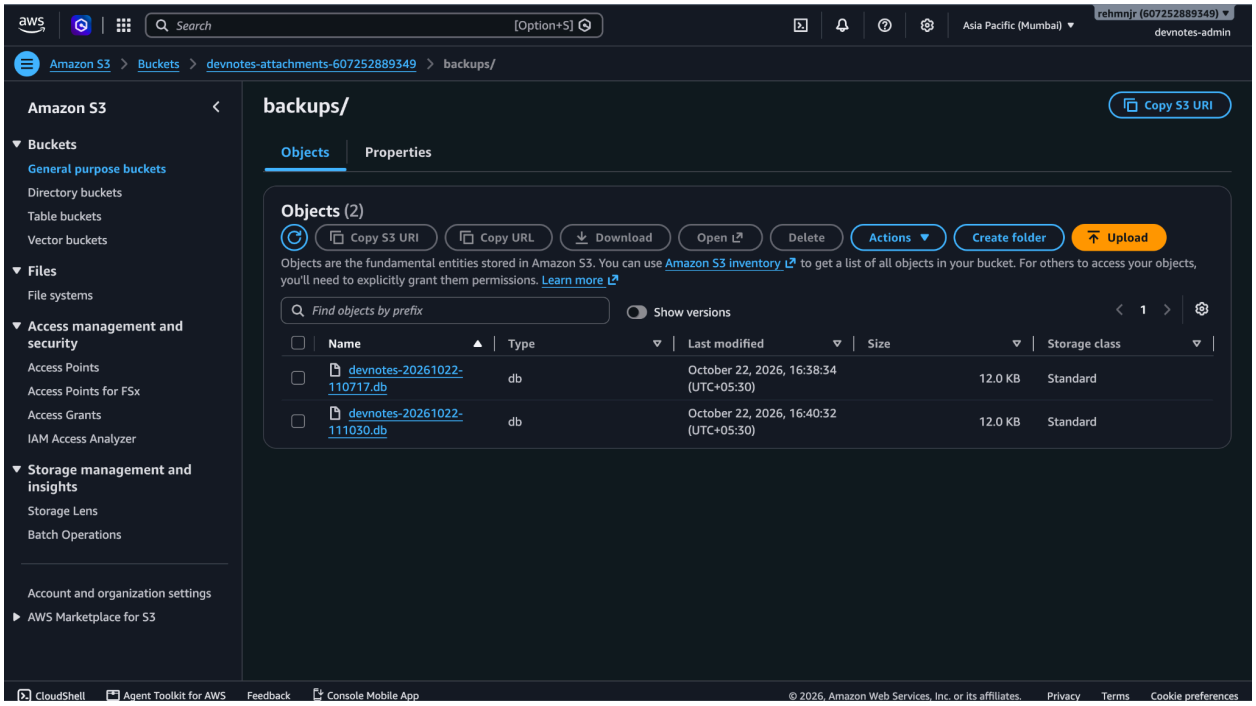
Configuration:

- **Bucket:** `devnotes-attachments-607252889349`
- **Prefix:** `backups/`
- **Backup files:** `devnotes-20261022-110717.db` and `devnotes-20261022-111030.db`

- **Size:** 12 KB each
- **Storage class:** S3 Standard

Result: Two backup files were present in S3. One file was downloaded to the local machine, and its SQLite integrity check returned `ok`.

Important note: These `.db` files are SQLite backups. Since DevNotes was migrated to MySQL on Amazon RDS, the SQLite backup should not be treated as a verified backup of the latest RDS database.




```
rehmnjr@MacJr DevNotes % ls -lh ~/Downloads/devnotes-20261022-110717.db
-rw-r--r--@ 1 rehmnjr  staff    12K Oct 23 23:56 /Users/rehmnjr/Downloads/devnotes-20261022-110717.db
rehmnjr@MacJr DevNotes % sqlite3 ~/Downloads/devnotes-20261022-110717.db "PRAGMA integrity_check;"
ok
rehmnjr@MacJr DevNotes %
```

```
rehmnjr@MacJr DevNotes % ls -lh ~/Downloads/devnotes-20261022-110717.db
-rw-r--r--@ 1 rehmnjr  staff    12K Oct 23 23:56 /Users/rehmnjr/Downloads/devnotes-20261022-110717.db
rehmnjr@MacJr DevNotes %
```

Amazon RDS — MySQL Database Setup

Configuration

- **AWS Service:** Amazon RDS
- **DB Identifier:** `database-1`
- **Database Engine:** MySQL-compatible engine
- **Status:** Available
- **Instance Role:** Instance
- **Availability Zone:** `ap-south-1b` (Mumbai Region)
- **Instance Class:** `db.t4g.micro` (as configured in the project)

What did I do?

We created a managed MySQL database using Amazon RDS for the DevNotes application. The database was configured to run in the Mumbai AWS Region.

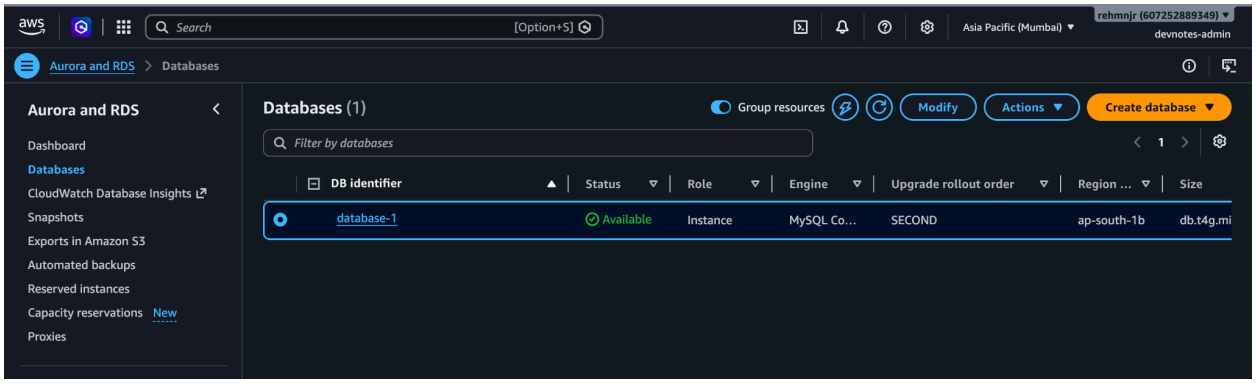
Why did I use RDS?

Amazon RDS manages database infrastructure and supports automated database maintenance and backups. It separates application compute from database storage, making the architecture easier to manage.

Result

The RDS database `database-1` is shown as **Available**, indicating that the database instance is ready for connections.

The DevNotes backend was configured to connect to this MySQL database instead of relying solely on its local SQLite database.



Amazon RDS — Connectivity & Security Configuration

1. Database Details

- **DB Identifier:** `database-1`
- **Engine:** MySQL Community
- **Instance Class:** `db.t4g.micro`
- **Status:** Available
- **AWS Region:** Asia Pacific (Mumbai)
- **Availability Zone:** `ap-south-1b`

2. Connection Configuration

- **Database Name:** `mysql` (as displayed in the connection panel)
- **Master Username:** `devnotesadmin`
- **Port:** `3306`
- **Internet Access Gateway:** Disabled
- **Endpoint:** `database-1.cluugk4m820m.ap-south-1.rds.amazonaws.com`

3. Networking Configuration

- **VPC:** `DevNotes-VPC-vpc`
- **VPC ID:** `vpc-0f132930898a5967e`
- **DB Subnet Group:** `default-vpc-0f132930898a5967e`
- **Network Type:** IPv4
- **Security Group:** `DevNotes-DB-SG`
- **Publicly Accessible:** No

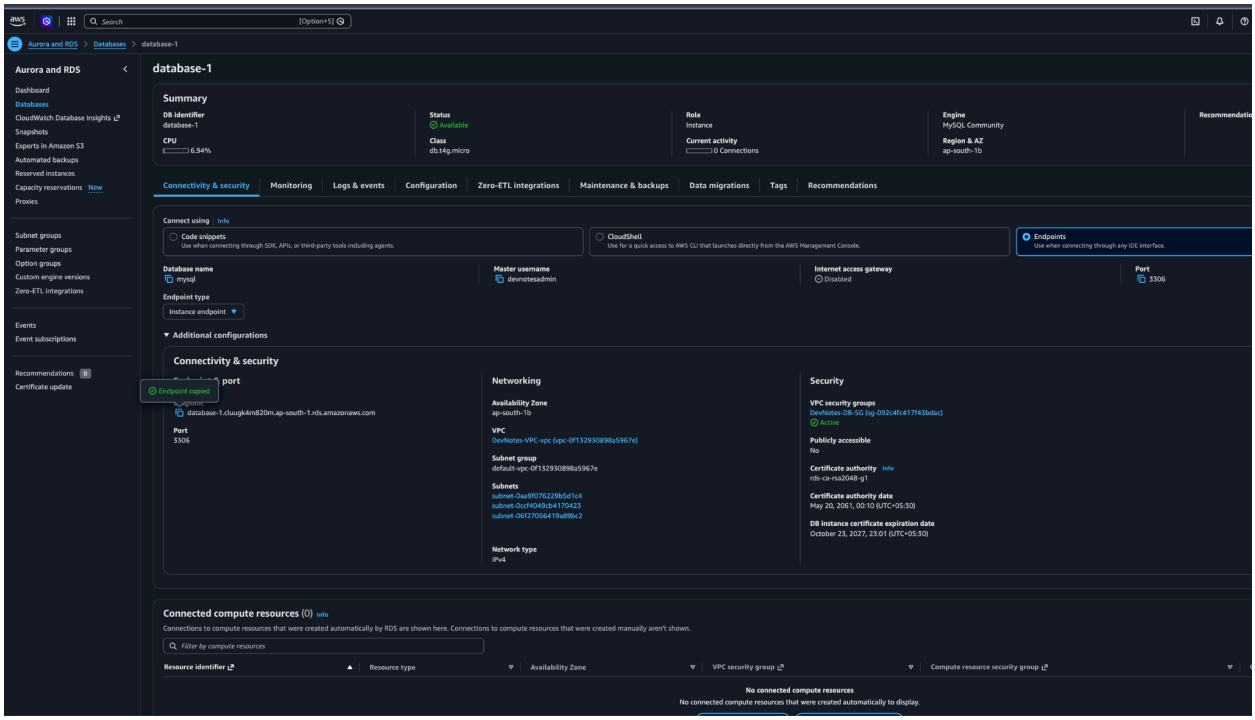
4. Security Explanation

The RDS database is configured as a private database and is not publicly accessible. The application server connects to it through the VPC network.

The database security group controls inbound database traffic on port `3306`. In our project, the intended configuration allows MySQL traffic from `DevNotes-Web-SG`, rather than opening the database port to everyone.

5. Project Outcome

The DevNotes backend was configured to use Amazon RDS MySQL for persistent data storage. This separates the database layer from the EC2 application server and provides a more structured cloud architecture.


Search
[Option+S]
Aurora and RDS
Databases
database-1
Aurora and RDS
Dashboard
Databases
CloudWatch Database Insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Capacity reservations New
Proxies
Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations
Events
Event subscriptions
Recommendations 0
Certificate update
database-1
Summary
DB identifier
database-1
Status
Available
Role
Instance
Engine
MySQL Community
CPU
6.94%
Class
db.t4g.micro
Current activity
0 Connections
Region & AZ
ap-south-1b
Connectivity & security
Monitoring
Logs & events
Configuration
Zero-ETL integrations
Maintenance & backups
Data migrations
Tags
Recommendations
Connect using Info
Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.
CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
Endpoints
Use when connecting through any IDE interface.
Database name
mysql
Master username
devnotesadmin
Internet access gateway
Disabled
Port
3306
Endpoint type
Instance endpoint
Additional configurations
Connectivity & security
port
Endpoint copied
database-1.cluugk4m820m.ap-south-1.rds.amazonaws.com
Port
3306
Networking
Availability Zone
ap-south-1b
VPC
DevNotes-VPC-vpc (vpc-0f132930898a5967e)
Subnet group
default-vpc-0f132930898a5967e
Subnets
subnet-0aa9f07622f6b5d1c4
subnet-0ccf4045fcb4170423
subnet-06f27056419a89bc2
Network type
IPv4
Security
VPC security groups
DevNotes-DB-SG (sg-092c4fc417f43bdac)
Active
Publicly accessible
No
Certificate authority Info
rds-ca-rsa2048-g1
Certificate authority date
May 20, 2061, 00:10 (UTC+05:30)
DB instance certificate expiration date
October 23, 2027, 23:01 (UTC+05:30)
Connected compute resources (0) Info
Connections to compute resources that were created automatically by RDS are shown here. Connections to compute resources that were created manually aren't shown.
Filter by compute resources
Resource identifier
Resource type
Availability Zone
VPC security group
Compute resource security group
No connected compute resources
No connected compute resources that were created automatically to display.